

Java Interface In Depth

Page No. :-

Date :- / /

- What is Interface and How to define
- Why we need Interface
- Methods in Interface
- Fields in an Interface
- Interface Implementation
- Nested Interface
- Difference between Interface and abstract class.

Java8 Interface features:

- Default method
 - How to handle the use-case when, class implements multiple interfaces have same name for the default method.
- Static method
- Functional Interface.

Java9 Interface feature:

- Private method.

Q. What is interface?

→ Interface is something which helps 2 System to interact with each other, without one system has to know the details of other.

or, in simple term I can say, it helps to achieve abstraction.

How to define the interface?

- Interface declaration consist of
 - Modifiers
 - "Interface" keyword
 - Interface Name
 - Comma Separated list of parent interfaces.
 - Body.

Only public and default Modifiers are allowed
(Protected and private are not allowed)

Ex :

```

public interface Bird {
    public void fly(); // body
}

// default interface
interface Animal {
    public void sound();
}
  
```

Comma Separated list of parent interfaces
(It can extend from class) Example :

```

public interface NonflyingBirds extends Birds, LivingThings {
    public void canRun();
}
  
```

multiple interfaces

Q Why we need Interface?

→ 1. Abstraction:

→ Using Interface, we can achieve full Abstraction means, we can define what class must do, but not how it will do.

Ex:

```

public interface Bird {
    public void fly();
}
  
```

 } what class do

How class do define here

```
public class Eagle implements Bird {
```

```
    @Override
```

```
    public void fly() {
```

```
        // the complex process of flying take place here  
        System.out.println("Eagle fly Implementation");  
    }
```

2. Polymorphism :

- Interface can be used as a Data Type.
- we can not create the object of an interface, but it can hold the reference of all the classes which implements it. And at runtime, it decide which method need to be invoked.

```
Ex: public class Hen implements Bird {
```

```
    @Override
```

```
    public void fly() {
```

```
        System.out.println("Hen fly Implementation");  
    }
```

```
}
```

```
public class Main {
```

```
    public static void main (String args[]) {
```

```
        Bird birdObj1 = new Eagle();
```

```
        Bird birdObj2 = new Hen();
```

```
        birdObj1.fly(); // Dynamically at runtime decide
```

```
        birdObj2.fly(); // which class fly() method invoke.
```

```
    }
```

```
}
```

Output : Eagle fly Implementation
Hen

3. Multiple Inheritance:

→ In Java Multiple Inheritance is possible only through Interface only.

```
public class WaterAnimal {
    public boolean canBreathe() {
        return true;
    }
}
```

```
public class LandAnimal {
    public boolean canBreathe() {
        return true;
    }
}
```

```
public class Crocodile extends LandAnimal, WaterAnimal {
}
```

```
public class Main {
    Psvm() {
```

```
        Crocodile obj = new Crocodile();
        Syst(obj.canBreathe());
    }
}
```

Note: If Java allowed multiple inheritance both LandAnimal and WaterAnimal define the same method canBreathe(). When crocodile calls obj.canBreathe(), the compiler cannot determine which implementation to execute. This creates method ambiguity (Diamond Problem) leading to compile time confusion.

Now Diamond problem due to multiple inheritance can be solved with help of Interface

```
Ex: public interface WaterAnimal {
    public boolean canBreathe();
}

public interface LandAnimal {
    public boolean canBreathe();
}
```



```
public class Crocodile implements WaterAnimal,
                                   LandAnimal {
```

```
    @Override
```

```
    public boolean canBreathe() {
```

```
        return true;
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Crocodile obj = new Crocodile();
```

```
        obj.canBreathe();
```

```
    }
```

```
}
```

- Both Interfaces declare the same method
- No method body is provided, so there is no conflict
- Crocodile implements both interfaces
- It must override canBreathe().
- Now the compiler knows exactly which method to call.
- No ambiguity exists.

Note: Java does not support multiple inheritance with classes because it can create method ambiguity.

Java solves this using Interfaces. Even if multiple interfaces declare the same method, the implementing class must override it, so the compiler always knows which implementation to execute. Hence the Diamond problem is avoided.
(No ambiguity)

Methods In Interface:

- All methods are implicit public only.
- Method can not be declare as final.

↓ method means no one can override but interface method definition is provided by child class. That's why.

public interface Bird {

← this is default method but internally it is treated as public whether you define public or not.

public void fly();
public void hasBeak();

Fields In Interface:

- fields are public, static and final implicitly (const)
- you can't make field private or protected.

public interface Bird {

public static final int MAXHEIGHTINFEET = 2000;

↑ by default

==

public interface Bird {

public static final int MAXHEIGHTINFEET = 2000;

Interface Implementation:

- Overriding method can not have more restrict access specifiers.
- Concrete class must override all the methods declared in the interface.
- Abstract classes are not forced to override all the methods
- A class can implement from multiple interfaces.

— Example of Abstract class Implementation of Interface:

```
public interface Bird {
    public void canfly();
    public void noOflegs();
}
```

```
public abstract class Eagle implements Bird {
    @Override
```

```
    public void canfly() {
        // Implementation goes here.
    }
```

```
    public abstract void beaklength();
}
```

```
public class WhiteEagle extends Eagle {
    @Override
```

```
    public void noOflegs() {
        // Implement interface method
    }
```

```
    @Override
```

```
    public void beaklength() {
```

```
        // Implementing abstract class method
    }
```

```
}
```

very less used

Nested Interface:

- Nested interface declared within another interface
- Nested interface declared within a class.

Generally its used to group, logical related interfaces. And Nested Interface.

Concrete class
↓
All abstract method
definition has to
+ here

Rules:

- A nested Interface declared within an Interface must be public.
- A nested Interface declared within a class can have any access modifier.
- When you implement outer Interface, inner Interface implementation is not required and vice versa.

Ex: Interface within Interface

```
public interface Bird {
    public void canFly();
```

Nested
inner Interface

```
    public interface NonFlyingBird {
        public void canRun();
    }
```

```
}
```

```
public class Eagle implements Bird {
    @Override
```

```
    public void canFly() {
```

```
        // Implementation goes here.
```

```
    }
```

```
}
```

```
public class Eagle implements Bird, NonFlyingBird {
    @Override
```

```
    public void canRun() {
```

```
        // code.
```

```
    }
```

```
}
```

Implement outer
Interface so
require to define
only abstract method
definition of
outer Interface only.


```

public class Eagle implements Bird, Bird.NonFlyingBird {
    @Override
    public void canRun() {
        // implementation
    }

    @Override
    public void canFly() {
        // code
    }
}

```

```

public class Main {
    public static void main() {
        Bird.NonFlyingBird obj = new Eagle();
        obj.canRun();
    }
}

```

Ex: Interface within a class

```

public class Bird {
    protected interface NonFlyingBird {
        public void canRun();
    }
}

```

Here interface is declared protected / private / public / default
 anything since class members can be anything
 so it is applied in interface also

```

public class Eagle implements Bird.NonFlyingBird {
    @Override
    public void canRun() {
        // code
    }
}

```


Interface

vs

Abstract class

- | | |
|--|---|
| 1. Keyword used here is "Interface" | Keyword is "abstract" |
| 2. child classes need to use keyword "implements" | child classes use keyword "extends" |
| 3. It can only have abstract method (from Java8 onwards it can have default, static and private method too, where we can provide implementation) | It can have both abstract and concrete method (non-abstract method) |
| 4. It can only extends from outer interfaces | It can extend from Another class and multiple interface. |
| 5. Variables are by default CONSTANTS | Variables can be static, final, non static, non final etc. |
| 6. Variables and Methods are by default public (In Java9, private method is supported) | Variables & Methods can be private, protected, default, public. |
| 7. Multiple Inheritance supported with this In Java. | Multiple inheritance not supported. |
| 8. It Can't provide implementation of any other interface or abstract class. | It can provide the implementation of the interface |
| 9. It can't have constructor | It can have constructor |
| 10. No need for any keyword to make method abstract, and by default is public | To declare the method abstract, we have to use "abstract" keyword and it can be protected, public, and default. |